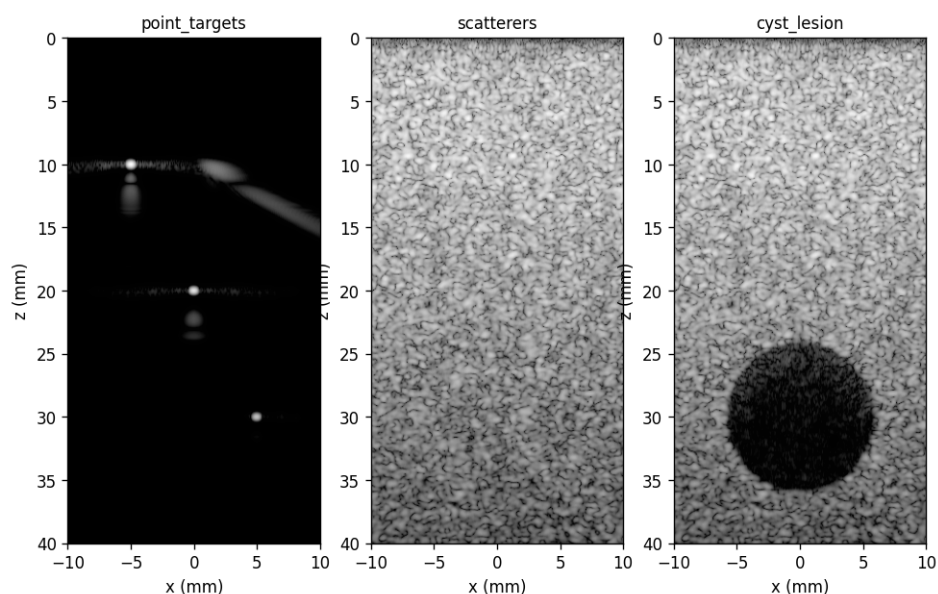


NITK-UsoundSim

Theory Guide: the physics and signal processing behind every team's task

A 2-D ultrasound simulator that turns a tissue phantom into a B-mode image, built by seven teams. Probe: **Philips L12-4 (FUS4103) reference model**, a 128-element linear array with 0.30 mm pitch at 8 MHz.

NITK-UsoundSim final images (50 dB, guided despeckle)



The simulator's final output: point targets, speckle, and an anechoic cyst.

Who this guide is for

Every team member. You do not need a signal-processing background: each idea is explained in plain words first, then the equation our code really uses, then a worked example with our real numbers.

Companion documents: **NITK-UsoundSim_Project_Flow.pdf** (who hands what to whom), **MODULES.md** (technical reference for each module), and **demo.ipynb** (runs every stage with plots).

Contents

How to read this guide	4
Team numbering used in this guide	4
Symbols and the values we use	5
1 Ultrasound in ten minutes	6
1.1 Sound wave, frequency and wavelength	6
1.2 Echo time tells depth	6
1.3 Why echoes happen: acoustic impedance	6
1.4 Attenuation and decibels	7
1.5 From echoes to a picture	7
1.6 Resolution: how small a thing we can see	8
2 Shared settings (config.py)	9
2.1 The RF time axis and the “time-axis rule”	9
3 T1 · Transducer	10
3.1 Theory in simple words	10
3.2 The equations	10
3.3 The $\lambda/2$ rule and grating lobes	10
3.4 T1's own code	11
4 T2 · Transmit beamforming	13
4.1 The transmit pulse	13
4.2 Focusing delays	13
4.3 Apodization (weighting the elements)	14
4.4 What our imaging chain uses, and why	14
5 T3 · Acoustic propagation	16
5.1 Time of flight	16
5.2 Attenuation	16
5.3 Forward and return propagation	17
5.4 Linear superposition	17
5.5 Sampling	17
6 T4 · Tissue interaction and RF generation	19
6.1 Scatterers: why tissue is modelled as many points	19
6.2 How one scatterer reflects	19
6.3 The three phantoms	19
6.4 Speckle: the grainy texture of ultrasound	20
6.5 From one echo to the full RF	21
7 T5 · Receive beamforming (delay-and-sum)	22

7.1 The idea of delay-and-sum (DAS)	22
7.2 The delays	22
7.3 Dynamic aperture and the F-number	23
7.4 Resolution and side lobes	23
8 T6 · Image reconstruction	25
8.1 Envelope detection with the Hilbert transform	25
8.2 Log compression	25
9 T7 · B-mode image formation	27
9.1 Dynamic range and gray mapping	27
9.2 Scan conversion	27
10 T6 · Post-image processing	29
10.1 Why not just blur?	29
10.2 The guided filter (He, Sun and Tang, 2013)	29
10.3 Our settings and their effect	29
11 Measuring image quality	32
11.1 Our final results	32
12 Parameter tests: theory checked by experiment	34
13 Assumptions and limitations	35
14 Glossary	36
15 Formula sheet	37
16 References and further reading	38

How to read this guide

Colour code, team numbering, symbols

Each team chapter follows the same pattern, so you can jump straight to your own part:

- **What this team does:** one sentence.
- **Theory in simple words:** the idea, without equations first.
- **The equations:** exactly the ones in our code, with every symbol explained.
- **Worked example:** the equation filled in with our real numbers.
- **How our code does it:** the steps and the file name.
- **Input → output** and a figure from our own simulation.
- **Check yourself:** short questions a teacher might ask, with answers.

Colour-coded boxes

Box	Meaning
Blue	Key idea: the one thing to remember
Green	Worked example with our numbers
Grey	How our code does it (file and steps)
Orange	Check yourself: viva-style questions

Team numbering used in this guide

This guide uses the same team split as the project-flow PDF. The code folders keep the original plan numbering. The table shows where each stage's code lives.

Team	Stage	Code (folder / file)
config	Shared settings	simulator/config.py
T1	Transducer (array geometry)	teams/T1_Transducer (piezo registry); geometry via T3's create_linear_array()
T2	Transmit beamforming	teams/T2_TX_Beamforming/Usound/src/beamformer.py
T3	Acoustic propagation	teams/T3_Acoustic_Propagation/usound_sim/nitk_usoundsim/acoustic_propagation.py
T4	Tissue interaction + RF generation	teams/T4_Tissue_Interaction: tissue_interaction.py, phantom.py, tissue_phantoms.py
T5	Receive beamforming (DAS)	teams/T5_RX_Beamforming/receive_beamforming.py
T6	Image reconstruction: envelope + log compression	teams/T6_B_mode/bmode_formation.py
T7	B-mode image formation: 50 dB gray map + scan conversion	bmode_formation.py (gray map) + teams/T7_.../post_processing.py (scan_convert)
T6	Post-image processing: despeckling (guided filter)	teams/T7_Image_Recon_Post_Processing/post_processing.py (postprocess)

Symbols and the values we use

Symbol	Meaning	Our value
c	speed of sound in tissue	1540 m/s
f_0	centre (transmit) frequency	8 MHz (probe range 4–12 MHz)
$\lambda = c / f_0$	wavelength	0.1925 mm
N	number of array elements	128
p (pitch)	centre-to-centre element spacing	0.30 mm (= 1.56λ)
w	element width	0.30 mm (= pitch, no gap assumed)
$D = N \cdot p$	aperture (array length)	38.4 mm
f_s	sampling frequency	40 MHz (5 samples per 8 MHz cycle)
α_0	attenuation coefficient	0.5 dB / (MHz·cm)
DR	display dynamic range	50 dB
$F\#$	receive F-number (depth / aperture width)	1.0
x, z	lateral position, depth	image: $x = \pm 10$ mm, $z = 0$ –40 mm
t_{axis}	RF time axis	3574 samples, starts at $-0.75 \mu\text{s}$

1 Ultrasound in ten minutes

The physics every team needs

Ultrasound imaging works like an echo in a valley. The probe sends a very short burst of sound into the body. Every small structure the sound hits sends a tiny echo back. The probe listens, and the **time** an echo takes to return tells us **how deep** the structure is. The **strength** of the echo tells us how bright to draw it.

1.1 Sound wave, frequency and wavelength

Sound is a pressure wave: tissue is squeezed and stretched many times per second. The number of squeezes per second is the **frequency f** (Hz). The distance between two squeezes is the **wavelength λ** . In soft tissue sound travels at about **$c = 1540$ m/s**.

$$\lambda = \frac{c}{f_0} = \frac{1540 \text{ m/s}}{8 \times 10^6 \text{ Hz}} = 0.1925 \text{ mm}$$

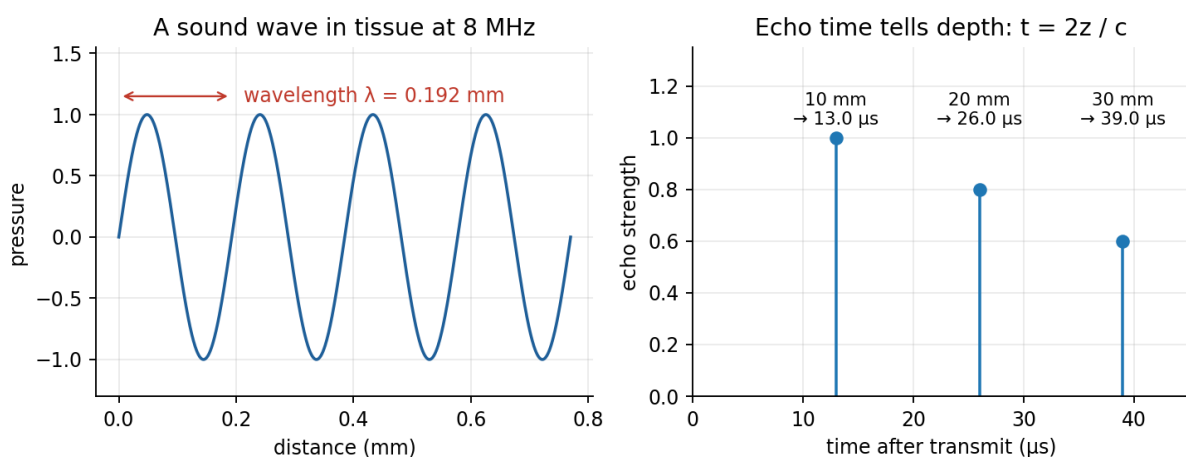
Key idea: smaller wavelength = finer detail

Higher frequency gives a smaller λ and therefore sharper images, but high frequencies are absorbed faster, so they cannot see deep. The L12-4 (4–12 MHz) is a high-frequency probe for shallow organs such as the breast, thyroid and blood vessels.

1.2 Echo time tells depth

The pulse travels down to depth z and the echo travels back up, so the sound covers $2z$ in total:

$$t = \frac{2z}{c} \quad \Leftrightarrow \quad z = \frac{ct}{2}$$



Left: an 8 MHz wave in tissue. Right: echoes from 10, 20 and 30 mm arrive after 13.0, 26.0 and 39.0 μs .

1.3 Why echoes happen: acoustic impedance

Each tissue has an **acoustic impedance $Z = \rho \cdot c$** (density $\times$ speed of sound), measured in MRayl. When sound meets a change in Z , part of it is reflected. The bigger the change, the stronger the echo. Soft tissue is about 1.63 MRayl in T4's model.

$$R = \frac{Z_2 - Z_1}{Z_2 + Z_1}$$

pressure reflection coefficient (T4's pressure_reflection_coefficient)

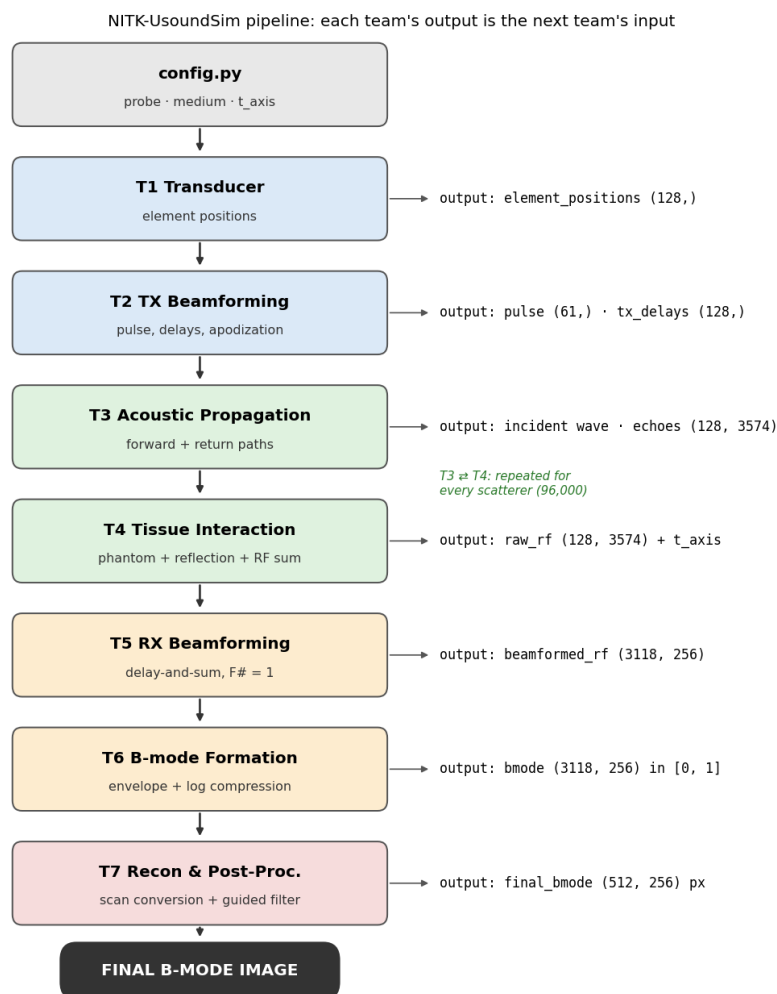
1.4 Attenuation and decibels

Tissue absorbs sound as it travels. The loss grows with distance **and** with frequency. Ultrasound people measure ratios in **decibels (dB)**: -6 dB is half the amplitude, -20 dB is one tenth, -40 dB is one hundredth.

$$\text{dB} = 20 \log_{10} \left(\frac{A}{A_{\text{ref}}} \right)$$

1.5 From echoes to a picture

- The probe has many small elements side by side (a **linear array**). Each element records its own echo signal over time: the **RF signal** (radio-frequency, because it oscillates at MHz).
- **Beamforming** combines the elements' signals so that we listen to one point at a time.
- Each vertical **scan line** becomes a column of the image. Echo strength is shown as brightness: **B-mode** (Brightness mode).
- A few more steps (envelope, log compression, gray mapping, scan conversion, despeckling) make it a clean picture.



The whole pipeline. Each team's output is the next team's input.

1.6 Resolution: how small a thing we can see

- **Axial resolution** (along depth) depends on the **pulse length**: a shorter pulse separates two close points in depth.
- **Lateral resolution** (sideways) depends on how narrow the **beam** is: about $\lambda \times \text{F-number}$.
- Both are measured as the **−6 dB width** (full width at half maximum amplitude) of a point target's image.

Check yourself

Q. An echo arrives 26 μs after transmit. How deep is the reflector?

A. $z = c \cdot t / 2 = 1540 \times 26 \times 10^{-6} / 2 \approx 20 \text{ mm}$.

Q. Why not always use 12 MHz for the sharpest image?

A. Attenuation grows with frequency: 12 MHz loses 3 $\times$ more dB per cm than 4 MHz, so deep regions become too dark.

Q. What does −20 dB mean?

A. The amplitude is 1/10 of the reference.

2 Shared settings (config.py)

One source of truth for every team

What it does: keeps every number that more than one team needs (speed of sound, sampling rate, probe, image size) in one file, **simulator/config.py**. Teams import the values instead of typing their own copies, so two stages can never disagree about, for example, the speed of sound.

2.1 The RF time axis and the “time-axis rule”

The transmit pulse is centred at $t = 0$, but it starts a little earlier (its front edge). So the recorded RF starts at $t = -0.75 \mu\text{s}$ (four pulse-widths σ before the centre), not at 0. The axis has 3574 samples at 40 MHz.

Key idea: the time-axis rule

To read the RF at a time τ , our code always uses **np.interp(τ , t_{axis} , rf)** on the real time axis, never “sample index = $\tau \times f_s$ ”. The shortcut would ignore the $-0.75 \mu\text{s}$ start and put every target about 0.58 mm too deep at 8 MHz ($c \times 0.75 \mu\text{s} / 2$).

$$\Delta z = \frac{c}{2f_s} = \frac{1540}{2 \times 40 \times 10^6} = 0.0193 \text{ mm per RF sample}$$

Setting	Value	Why
Imaged width	$x = \pm 10 \text{ mm}$, 256 scan lines (0.078 mm apart)	well inside the 38.4 mm array
Displayed depth	0–40 mm	T3's field of view
RF recorded to	60 mm	T5's own test checks a target at 55 mm
Transmit	one broadcast shot (all delays 0)	one simulation serves all 256 lines
Receive window	Hann, $F\# = 1$	see Chapter 7
Element directivity	on	see Chapter 3

3 T1 · Transducer

Piezoelectric elements and the geometry of the linear array

What this team does: describes the probe: the material that converts electricity into sound, and where the 128 elements sit.

3.1 Theory in simple words

A transducer element is a small piece of **piezoelectric** material (for example the ceramic PZT-5H). Piezoelectric means: apply a voltage and it changes shape (it sends sound); squeeze it with sound and it produces a voltage (it receives). The same element is used for both.

A **linear array** puts many narrow elements in a row. Their spacing is the **pitch**, and the total length is the **aperture**. A wide aperture can focus more sharply; a small pitch avoids false copies of objects (grating lobes).

3.2 The equations

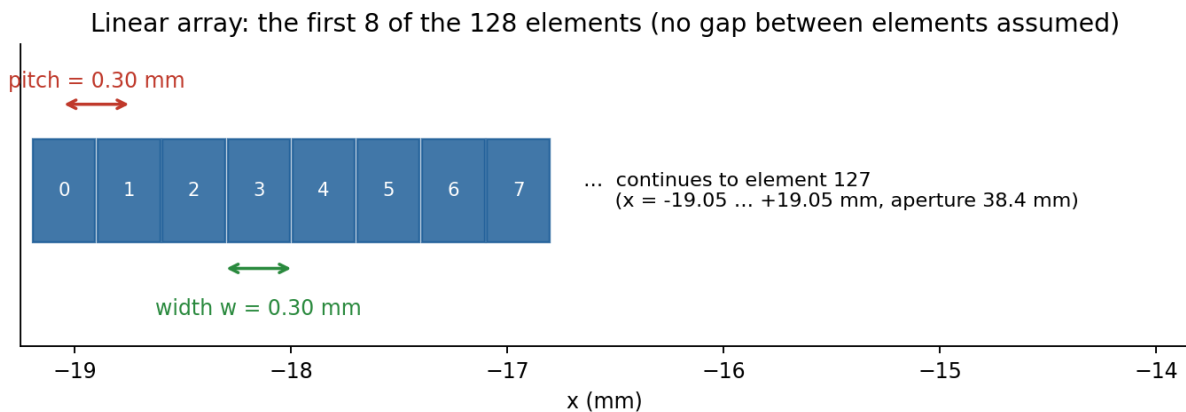
$$x_n = \left(n - \frac{N-1}{2} \right) p, \quad n = 0, 1, \dots, N-1$$

element centres, symmetric around $x = 0$ (create_linear_array)

Worked example with our numbers

$N = 128$, $p = 0.30$ mm $\rightarrow x_0 = -63.5 \times 0.30 = -19.05$ mm, $x_{127} = +19.05$ mm; aperture $D = 128 \times 0.30 = 38.4$ mm.

Pitch in wavelengths: $p / \lambda = 0.30 / 0.1925 = 1.56 \lambda$.



The first 8 of our 128 elements, numbered from 0. Each is 0.30 mm wide and they touch (no kerf assumed).

3.3 The $\lambda/2$ rule and grating lobes

If elements are more than half a wavelength apart, the array can be “fooled”: a strong echo coming from a steep angle arrives at neighbouring elements with a delay of exactly one full wave, so it looks like it came from straight ahead. This creates a false copy of the object called a **grating lobe**. It appears at the angle where

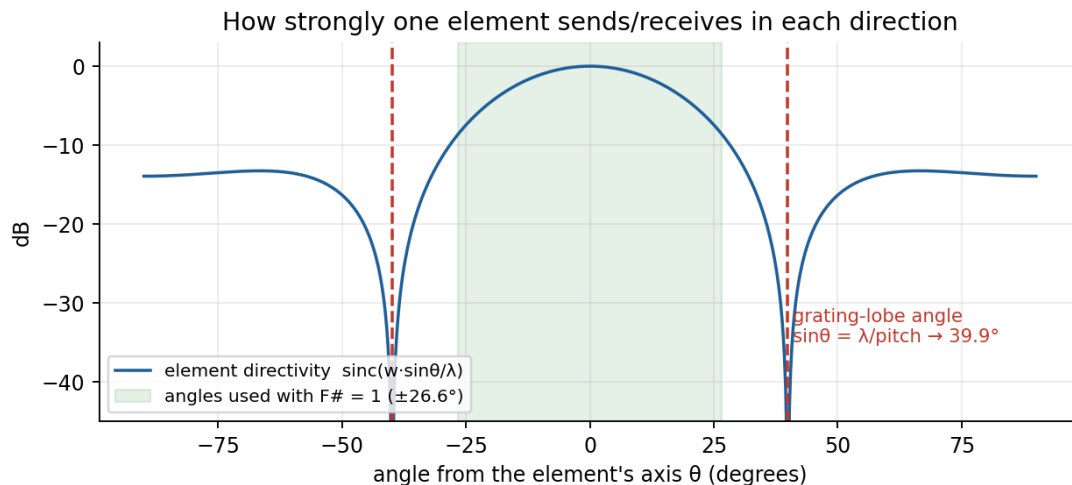
$$\sin \theta_g = \frac{\lambda}{p} = \frac{0.1925}{0.30} \Rightarrow \theta_g = 39.9^\circ$$

Our L12-4 model has $p = 1.56 \lambda$, well above $\lambda/2$, so grating lobes are a real risk. Two effects protect us:

- **Element directivity.** A real element of width w does not send and receive equally in all directions. It is strongest straight ahead and weaker at steep angles (a sinc pattern, as used in the Field II simulator).
- **F-number on receive** (Chapter 7) only listens up to $\pm 26.6^\circ$, below the 39.9° grating angle.

$$D_n(\theta) = \text{sinc}\left(\frac{w \sin \theta}{\lambda}\right), \quad \text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$$

applied on transmit and on receive in `tissue_phantoms.element_directivity()`



Worked example with our numbers

At the grating angle, $\sin \theta_g = \lambda/p$, so $D = \text{sinc}(w/p) = \text{sinc}(1) = 0$ at exactly 8 MHz: the element is “deaf” there.

The pulse also contains nearby frequencies (about 7.1–8.9 MHz at -6 dB), where the null is not exact, so ghosts are reduced, not removed.

Measured effect: without directivity the ghosts under each point target were only -11 to -13 dB; with it they are **-21 to -34 dB**.

3.4 T1's own code

How our code does it

T1 delivered a **piezo-material registry** (`teams/T1_Transducer/src`): a `Piezo` class that stores a material name and its frequency range, checks the range (both values numeric, non-negative, $\min \leq \max$), and a registry to add and look up materials such as Quartz and PZT-5H. `process_material()` returns a random frequency inside the material's range.

Because T1's code has no element positions, the array geometry comes from T3's `create_linear_array()`, called through `config.py`. T2's probe file names the material as **PZT-5H**, matching T1's registry.

Receives (input)	Does	Produces (output)	Code
<code>N_ELEMENTS = 128,</code> <code>PITCH = 0.30 mm (config)</code>	places elements symmetrically on the x-axis at $z = 0$	<code>element_positions</code> , shape (128,), metres	<code>config.get_element_positions()</code>

Check yourself

Q. What is the pitch of our array in wavelengths, and why does it matter?

A. 1.56λ . Above $\lambda/2$, grating lobes (false copies) can appear.

Q. At what angle would a grating lobe appear?

A. $\sin \theta = \lambda/p \rightarrow 39.9^\circ$.

Q. Why does the element's width help?

A. Its sinc directivity has a null at that same angle when $w = p$, so the element barely hears the grating direction.

Q. What does piezoelectric mean?

A. Voltage $\leftrightarrow$ mechanical deformation: the element both transmits and receives sound.

4 T2 · Transmit beamforming

The pulse, focusing delays and apodization

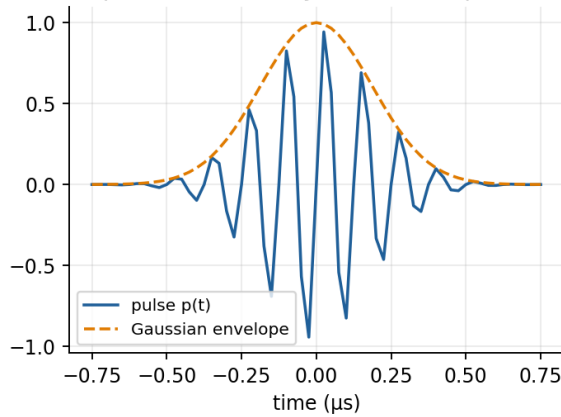
What this team does: decides **what** each element sends (the pulse) and **when** (the delays), so that the sound goes where we want.

4.1 The transmit pulse

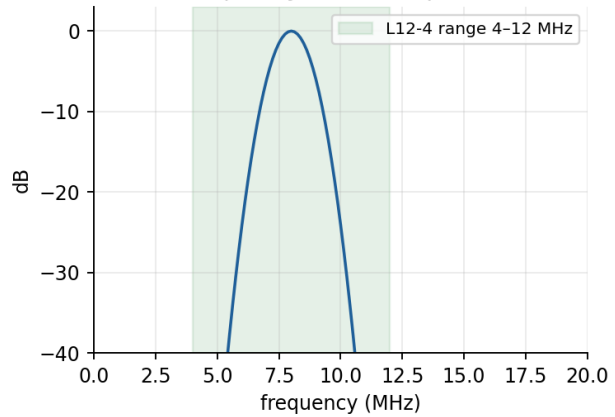
A short pulse gives good depth resolution, but it must still contain a few oscillations at the centre frequency. We use a sine wave at f_0 inside a smooth bell-shaped (Gaussian) envelope, 3 cycles long (T3's generate_pulse):

$$p(t) = \sin(2\pi f_0 t) \exp\left(-\frac{t^2}{2\sigma^2}\right), \quad \sigma = \frac{N_{\text{cycles}}}{2f_0} = \frac{3}{2 \times 8 \text{ MHz}} = 0.1875 \mu\text{s}$$

Transmit pulse: 8 MHz, 3 cycles, 61 samples at 40 MHz



Its frequency content (spectrum)



Our 8 MHz pulse (61 samples from -0.75 to $+0.75 \mu\text{s}$) and its spectrum. Its -6 dB band is 7.1–8.9 MHz.

Key idea: pulse length ↔ axial resolution ↔ bandwidth

Short pulse → wide spectrum (bandwidth) → fine axial resolution. The -6 dB width of the envelope in time is 2.355σ ; converted to depth (divide by 2 for the round trip):

$$\Delta z_{-6 \text{ dB}} = \frac{c}{2} \cdot 2.355 \sigma = 770 \times 2.355 \times 0.1875 \mu\text{s} = 0.340 \text{ mm}$$

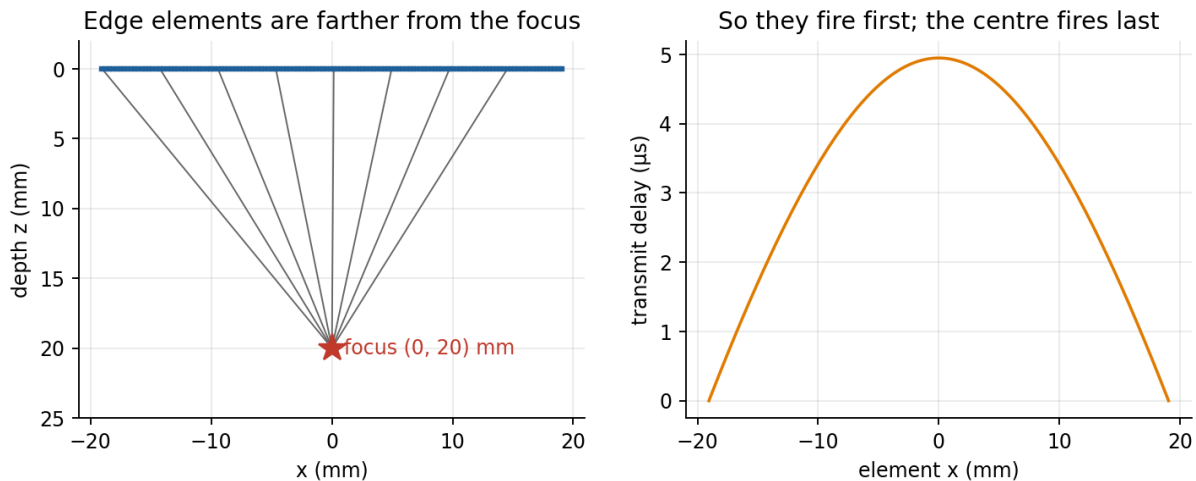
Measured on our point targets: **0.327–0.346 mm**. Theory and simulation agree.

4.2 Focusing delays

To focus at a point (x_f, z_f) , every element's wave must arrive there **at the same moment**. Edge elements are farther away, so they must fire **first**; the centre fires last:

$$d_n = \sqrt{(x_n - x_f)^2 + z_f^2}, \quad \tau_n = \frac{\max_n d_n - d_n}{c}$$

To **steer** the beam by an angle θ without focusing, a linear delay is used instead: $\tau_n = x_n \sin \theta / c$ (shifted so the smallest is 0).

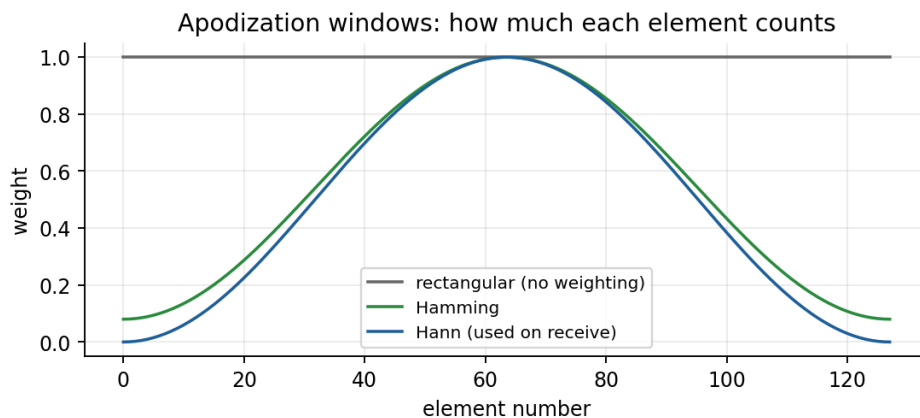


Focus at (0, 20) mm: the delay curve is a dome, largest at the centre (4.95 μs).

4.3 Apodization (weighting the elements)

If all elements send with equal strength (rectangular window), the beam has strong **side lobes**: weak copies of the beam beside the main one. Tapering the edges with a smooth window (Hann or Hamming) lowers the side lobes, but makes the main beam a little wider. This is the classic trade-off.

$$w_{\text{Hann}}(n) = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), \quad w_{\text{Hamming}}(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right)$$



The three windows compared in the parameter tests.

4.4 What our imaging chain uses, and why

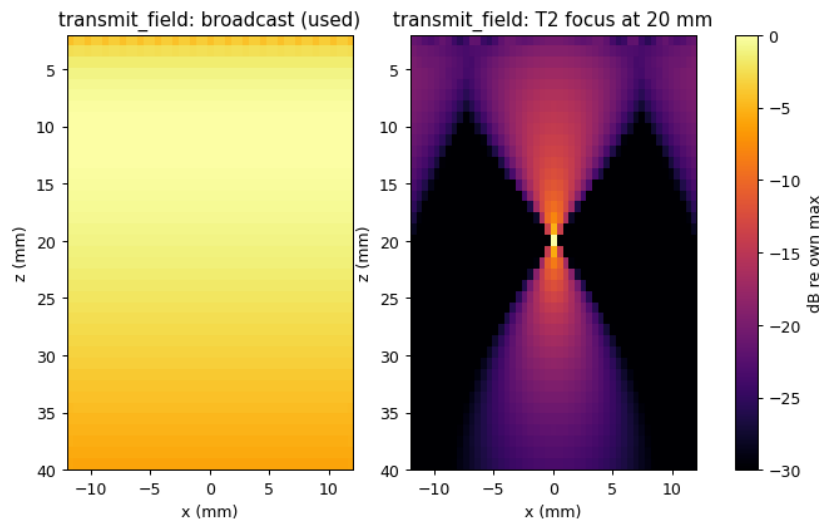
Key idea: one broadcast transmit

All 128 elements fire together (all delays 0). One transmit lights up the whole field, and one RF simulation serves all 256 scan lines. A focused transmit for every line would need 256 separate simulations of about 4 min each.

Focusing then happens on **receive** (T5), at every depth.

Worked example with our numbers

Parameter test: with T2's delays focused at 20 mm, the incident wave at a target at (0, 20) mm is **+18.36 dB** stronger than with the broadcast. With the focus at 10 mm it is +0.00 dB and at 30 mm +4.78 dB: focusing helps most at the target depth.



How our code does it

T2's beamformer.py: `create_linear_array()`, `make_apodization()` (rect / Hann / Hamming, normalised to max 1), `calculate_tx_delays()` (focus or steering) and `build_tx_package()`. T2's probe file (`config/transducer_interface.json`) is where the L12-4 values come from: 128 elements, 0.30 mm, 4–12 MHz, 8 MHz.

The imaging chain uses `config.get_tx_delays()` (zeros) and T3's pulse. T2's delays are used in the parameter tests and the demo notebook.

Receives (input)	Does	Produces (output)	Code
element_positions (T1), focus or angle, c	pulse, per-element delays, apodization	pulse (61,), tx_delays (128,) s, weights (128,)	T2 beamformer.py; T3 generate_pulse()

Check yourself

Q. Why do edge elements fire first when focusing?

A. They are farther from the focus, so they need a head start to arrive at the same time.

Q. What happens to side lobes and beam width with a Hann window?

A. Side lobes go down ($-18.2 \text{ dB} \rightarrow -32.7 \text{ dB}$ on receive in our test); the main beam widens ($0.30 \rightarrow 0.46 \text{ mm}$).

Q. Why don't we focus on transmit in the images?

A. One focused transmit per line would need 256 simulations; the broadcast needs one. Receive focusing does the rest.

Q. How long is our pulse?

A. 3 cycles, $\sigma = 0.1875 \mu\text{s}$, stored as 61 samples (-0.75 to $+0.75 \mu\text{s}$).

5 T3 · Acoustic propagation

How the pulse travels to a scatterer and back

What this team does: moves the pulse through tissue: from every element to a point (forward) and from that point back to every element (return), with the right delay and loss.

5.1 Time of flight

Sound goes in a straight line at constant speed c . The distance from element n at $(x_n, 0)$ to a scatterer at (x_s, z_s) is

$$d_n = \sqrt{(x_s - x_n)^2 + z_s^2}, \quad t_n = \frac{d_n}{c}$$

Worked example with our numbers

Target at (0, 20) mm: the centre elements are 20 mm away $\rightarrow 20 \text{ mm} / 1540 \text{ m/s} = \mathbf{12.99 \mu s}$; the edge elements at $\pm 19.05 \text{ mm}$ are $\sqrt{(19.05^2 + 20^2)} = 27.6 \text{ mm}$ away $\rightarrow \mathbf{17.94 \mu s}$. That spread of arrival times is the curve seen in the raw echoes.

5.2 Attenuation

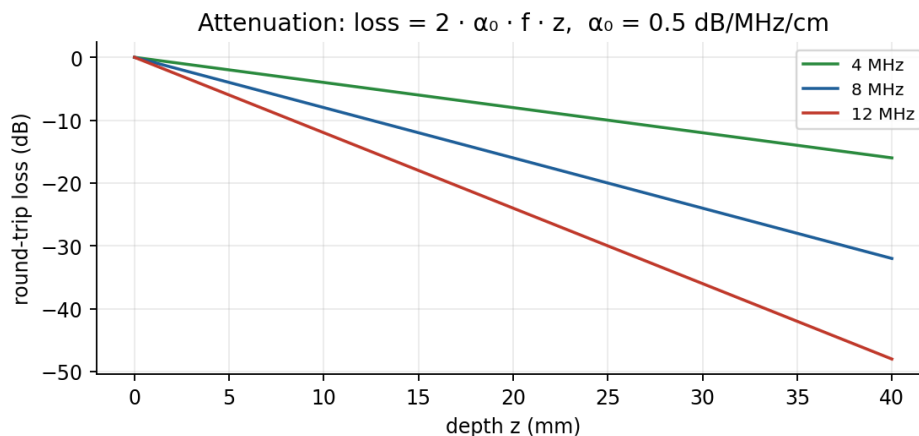
The amplitude drops exponentially with distance; in dB the loss is simply proportional to frequency $\times$ distance:

$$\text{loss}_{\text{dB}} = \alpha_0 f_0 [\text{MHz}] d [\text{cm}], \quad A(d) = 10^{-\text{loss}_{\text{dB}}/20}$$

Worked example with our numbers

At 8 MHz: $0.5 \times 8 = \mathbf{4 \text{ dB per cm}}$ one way. A target 2 cm deeper loses $2 \times 2 \times 4 = \mathbf{16 \text{ dB}}$ more (there and back).

Parameter test: targets at 10 and 30 mm, $\alpha_0 = 0.5 \rightarrow$ measured extra loss -16.11 dB (theory -16.00); $\alpha_0 = 1.0 \rightarrow -32.21 \text{ dB}$ (theory -32.00).



Round-trip attenuation versus depth for 4, 8 and 12 MHz.

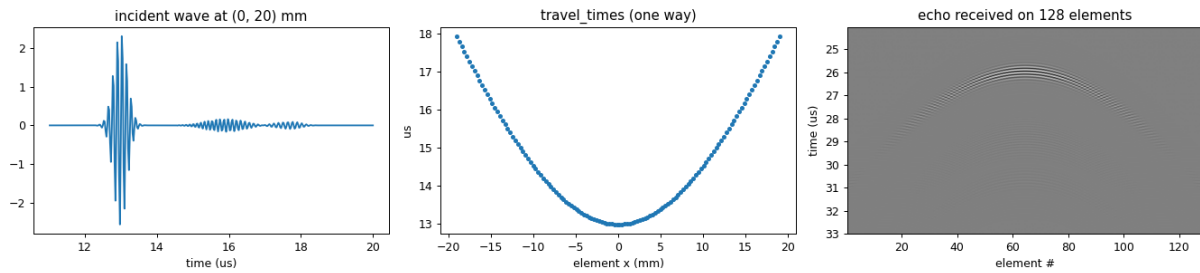
5.3 Forward and return propagation

Forward: the wave arriving at the scatterer is the sum of the pulses from all elements, each delayed by its own travel time and transmit delay, and weakened by its own path loss. **Return:** the scattered wave travels back to each element m with its own delay and loss.

$$\text{incident}(t) = \sum_n D_n A(d_n) p\left(t - \tau_{TX,n} - \frac{d_n}{c}\right)$$

$$rf_m(t) = D_m A(d_m) \text{ reflected}\left(t - \frac{d_m}{c}\right)$$

D = element directivity (Chapter 3), added by T4's tissue_phantoms.py around T3's functions



Incident wave at (0, 20) mm, one-way travel times from every element, and the echo received on all 128 elements.

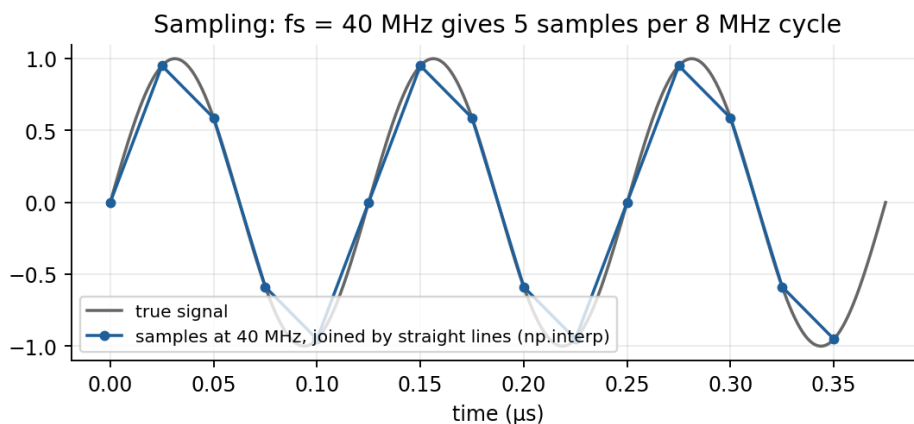
5.4 Linear superposition

Key idea: echoes simply add

The medium is linear: the echo from many scatterers is the sum of their separate echoes. So the simulator handles one scatterer at a time and adds up the results. This is also why the work can be split across CPU cores and summed at the end.

5.5 Sampling

The computer stores the RF only at discrete times, every $1/f_s = 25$ ns. The **Nyquist rule** says we need more than two samples per cycle of the highest frequency: $2 \times 12 \text{ MHz} = 24 \text{ MHz} < 40 \text{ MHz}$, so our sampling is enough. To read the signal **between** samples (delays are never exact multiples of 25 ns), the code joins samples with straight lines (linear interpolation, `np.interp`).



At 40 MHz an 8 MHz wave gets 5 samples per cycle; `np.interp` joins them with straight lines.

Linear interpolation slightly smooths the peaks (it acts as a weak low-pass filter). The parameter test shows the peak level is about 2.8 dB lower at 4 samples per cycle than at 8, but the **position** of the target does not move. Because every image is normalised to its own peak, this uniform loss

does not change the picture.

How our code does it

`forward_propagation(scatterer, element_x, t_axis, f0, tau_tx)`: distance → delay → attenuation → sum of delayed pulses.

`return_propagation(reflected, t_axis, scatterer, element_x, f0)`: shifts the reflected wave to every element with `np.interp`.

T3's functions are used unchanged; $f_0 = 8$ MHz is passed explicitly because T3's own default is 5 MHz.

Receives (input)	Does	Produces (output)	Code
element positions, tx_delays, pulse, one scatterer, t_axis	straight-ray delay + attenuation, forward and back	incident wave (3574,); echoes at the elements (128, 3574)	acoustic_propagation.py (T3)

Check yourself

Q. Why is the raw echo from one point a curve across the elements?

A. Edge elements are farther away, so the echo reaches them later (12.99 μs at the centre versus 17.94 μs at the edges for 20 mm).

Q. Is 40 MHz sampling enough for a 12 MHz signal?

A. Yes: Nyquist needs more than 24 MHz.

Q. How much signal do we lose from 10 mm to 30 mm at 8 MHz?

A. About 16 dB ($2\text{ cm} \times 2\text{ ways} \times 4\text{ dB/cm}$); measured 16.11 dB.

Q. What does “linear superposition” let us do?

A. Simulate each scatterer separately and add the echoes, in parallel on all CPU cores.

6 T4 · Tissue interaction and RF generation

Scatterers, speckle, the cyst, and the 128-channel RF

What this team does: builds the “patient” (a phantom made of point scatterers), decides how strongly each point reflects, and adds up all echoes into the raw RF data that a real scanner would record.

6.1 Scatterers: why tissue is modelled as many points

Real tissue is full of structures much smaller than a wavelength (cells, fibres). Each one reflects a tiny echo. We model tissue as many randomly placed point scatterers, each with a random strength. This is the standard approach of the Field II simulator.

$$x, z \sim \text{Uniform}, \quad \text{amp} \sim \mathcal{N}(0, \sigma^2 = 1)$$

T4's `generate_liver_phantom()`: uniform positions, Gaussian amplitudes (positive or negative)

Worked example with our numbers

Phantom area: $x = \pm 12$ mm (24 mm) $\times z = 0$ –40 mm. Density 1×10^8 scatterers/m² $\rightarrow 0.024 \times 0.040 \times 10^8 = \mathbf{96,000}$ scatterers.

That is about 15 scatterers per resolution cell (0.33 mm $\times 0.47$ mm $\times 10^8$ m⁻² ≈ 15.5), enough for “fully developed” speckle.

6.2 How one scatterer reflects

T4 multiplies the incident wave by a gain. If the scatterer carries its own impedance, the reflection coefficient R (Chapter 1) makes the echo stronger; an optional scale makes a region brighter or darker:

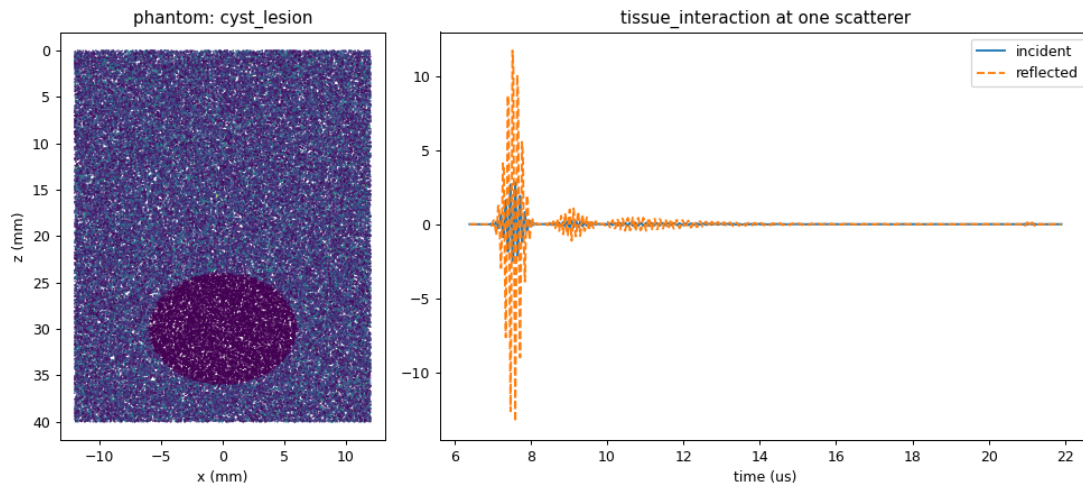
$$\text{reflected}(t) = G \cdot \text{incident}(t), \quad G = \text{amp} (1 + |R|) \text{ scale}$$

In our phantoms no local impedance is set, so $G = \text{amp} \times \text{scale}$.

6.3 The three phantoms

Phantom	What it is	What it tests
Point targets	3 points at (−5, 10), (0, 20), (+5, 30) mm, amp 1	position accuracy and resolution
Scatterers (speckle)	96,000 random scatterers	speckle texture and brightness uniformity
Cyst	same 96,000, with an anechoic circle $r = 6$ mm at (0, 30) mm (11,169 scatterers inside, scale 0.0)	contrast and edge sharpness

T4's code also supports lesion classes: **hypoechoic** (darker, scale 0.35), **isoechoic** (1.0) and **hyperechoic** (brighter, 1.8). Our cyst uses scale 0.0: **anechoic**, like a fluid-filled cyst that produces no echoes.



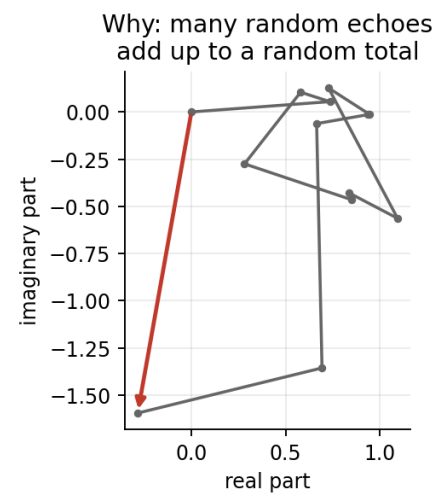
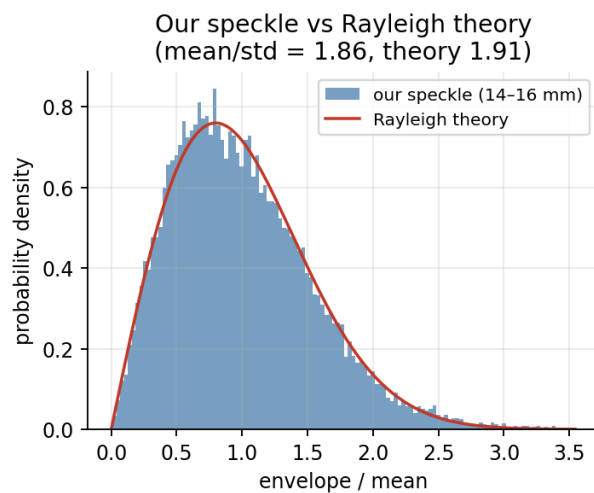
Left: the cyst phantom (colour = amplitude). Right: T4's tissue interaction at one scatterer.

6.4 Speckle: the grainy texture of ultrasound

Inside one resolution cell, many scatterers echo at the same time. Their echoes have random phases, so they add up sometimes strongly, sometimes cancelling. The result is a granular pattern called **speckle**. It is not noise from the electronics; it is an interference pattern, and it looks the same every time for the same tissue.

Theory (Burckhardt 1978; Wagner et al. 1983): with many random scatterers per cell, the envelope brightness follows a **Rayleigh distribution**, whose mean divided by its standard deviation is fixed:

$$\text{SNR}_{\text{speckle}} = \frac{\text{mean}}{\text{std}} = \sqrt{\frac{\pi}{4 - \pi}} = 1.91$$



Our speckle at 14-16 mm matches the Rayleigh curve (mean/std 1.86). Right: a random sum of echoes.

Worked example with our numbers

Measured over depth bands from 6 to 38 mm: **1.73-1.92** (theory 1.91). Our speckle is fully developed, as in real tissue.

6.5 From one echo to the full RF

$$rf_m(t) = \sum_{s=1}^{96\,000} echo_{m,s}(t)$$

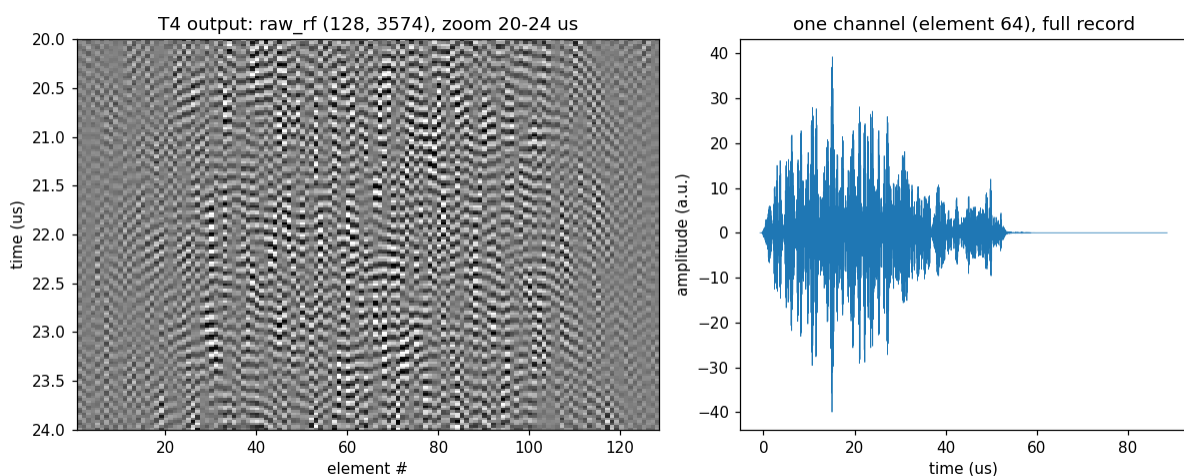
for every element m: add the echoes of all scatterers (linear superposition)

How our code does it

tissue_phantoms.py: build_phantom(kind) → list of {x, z, amp}.

For each scatterer: incident_wave() (T3 forward × directivity) → tissue_interaction() (T4) → return_propagation() (T3) × directivity.

simulate_rf() splits the scatterers into chunks and runs them in parallel processes on all CPU cores, then adds the chunks. About 3.6–4.4 min per 96,000-scatterer phantom on 8 cores.



T4's output raw_rf (128 × 3574): a 20–24 μs zoom of all channels, and one full channel. Speckle echoes overlap everywhere.

Receives (input)	Does	Produces (output)	Code
T3's propagation functions, phantom settings (config)	builds the phantom, reflects the wave at every scatterer, sums the echoes	raw_rf (128, 3574) + t_axis	tissue_interaction.py , phantom.py, tissue_phantoms.py

Check yourself

Q. Is speckle noise that a better amplifier could remove?

A. No. It is interference from many sub-wavelength scatterers; it is part of the signal.

Q. What value of mean/std do we expect for fully developed speckle?

A. 1.91 (Rayleigh). We measured 1.73–1.92.

Q. Why is our cyst black?

A. Scatterers inside have scale 0.0 (anechoic), so no echoes come from inside.

Q. How many scatterers, and how many inside the cyst?

A. 96,000 in total; 11,169 inside the 6 mm cyst.

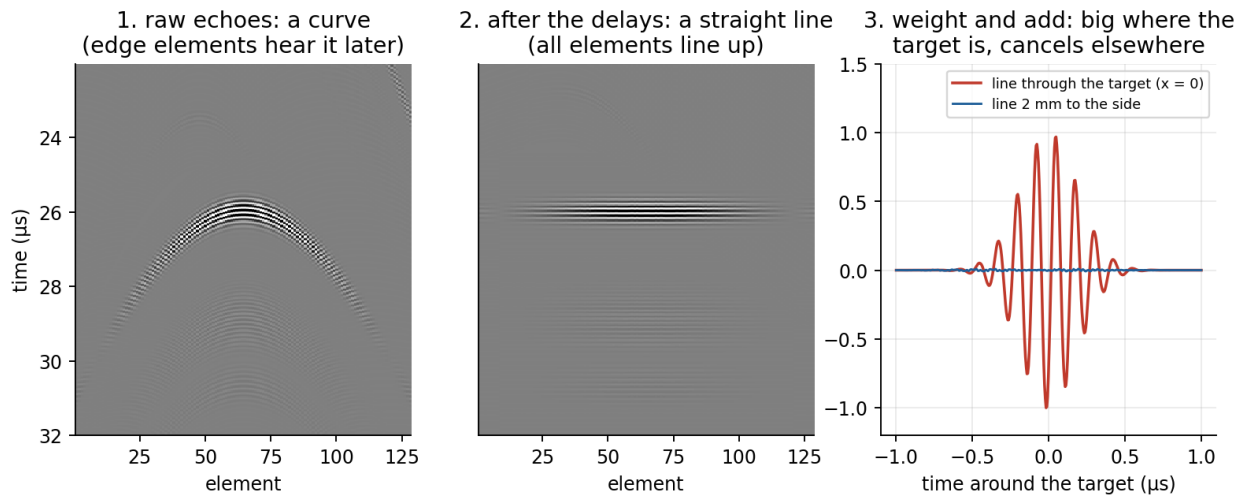
7 T5 · Receive beamforming (delay-and-sum)

Listening to one point at a time

What this team does: turns 128 channels of raw echoes into 256 focused scan lines. It is the heart of the scanner.

7.1 The idea of delay-and-sum (DAS)

An echo from a point reaches the elements at different times (a curve, Chapter 5). If we **delay** each channel by exactly its extra travel time, the echoes from that point line up. Then we **sum** them: echoes from the chosen point add up strongly, while echoes from other places are misaligned and cancel. Doing this for every depth and every line gives the image.



Real point-target RF from our simulation. Aligning and adding gives a strong pulse on the target line; 2 mm to the side it cancels (about -40 dB).

7.2 The delays

For a pixel at (x_l, z) on scan line l , the time the echo needs is the transmit time (the broadcast wave first reaches the point from the nearest element) plus the return time to element m :

$$\tau_{TX}(z) = \frac{\min_n \sqrt{(x_l - x_n)^2 + z^2}}{c}, \quad \tau_{RX,m}(z) = \frac{\sqrt{(x_l - x_m)^2 + z^2}}{c}$$

$$s_l(z) = \frac{\sum_m w_m(z) \text{rf}_m(\tau_{TX} + \tau_{RX,m})}{\sum_m w_m(z)}$$

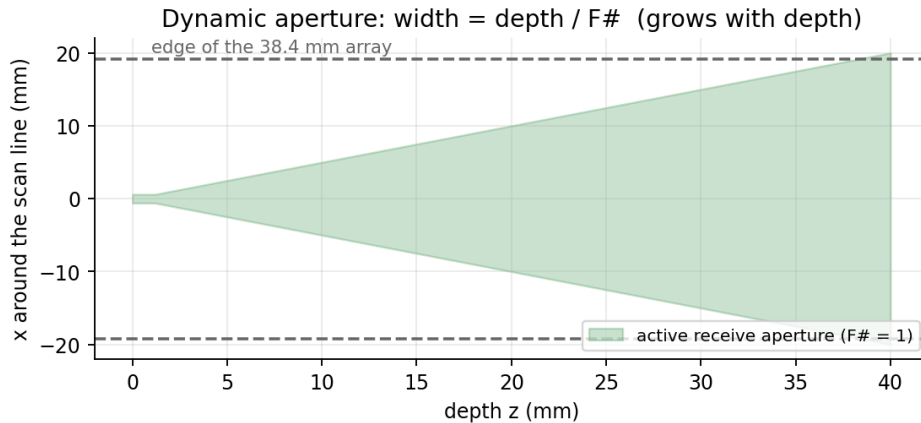
Because the delays are recomputed for every depth, the receive focus follows the echo down: **dynamic receive focusing**. Every lookup $\text{rf}_m(\tau)$ uses np.interp on the real t_axis (the time-axis rule).

7.3 Dynamic aperture and the F-number

The **F-number** is focal depth divided by aperture width. Near the surface only a few elements can usefully hear a point; deep down, more can. So the active aperture grows with depth, keeping F# constant:

$$a(z) = \max\left(\frac{z}{2F\#}, 2p\right), \quad w_m(z) = \cos^2\left(\frac{\pi |x_m - x_l|}{2a}\right) \text{ for } |x_m - x_l| < a$$

a = half-width of the active aperture; Hann-shaped weights centred on the line



With $F\# = 1$ the aperture is as wide as the depth: 10 mm wide at 10 mm, 40 mm (whole array) near 38 mm.

Key idea: why $F\# = 1$

With $F\# = 1$, the steepest angle any element listens at is $\text{atan}(1/2) = 26.6^\circ$, below the 39.9° grating angle. With the full fixed aperture, a grating-lobe streak appeared at -17 dB; with $F\# = 1$ it is **below -100 dB**. Cost: the lateral -6 dB width grew from 0.31 to 0.47 mm (fewer elements at shallow depth).

7.4 Resolution and side lobes

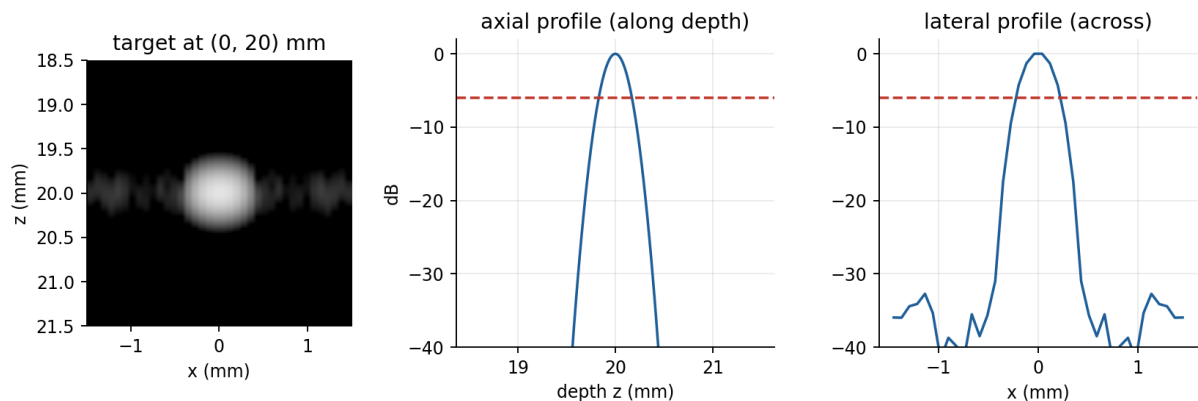


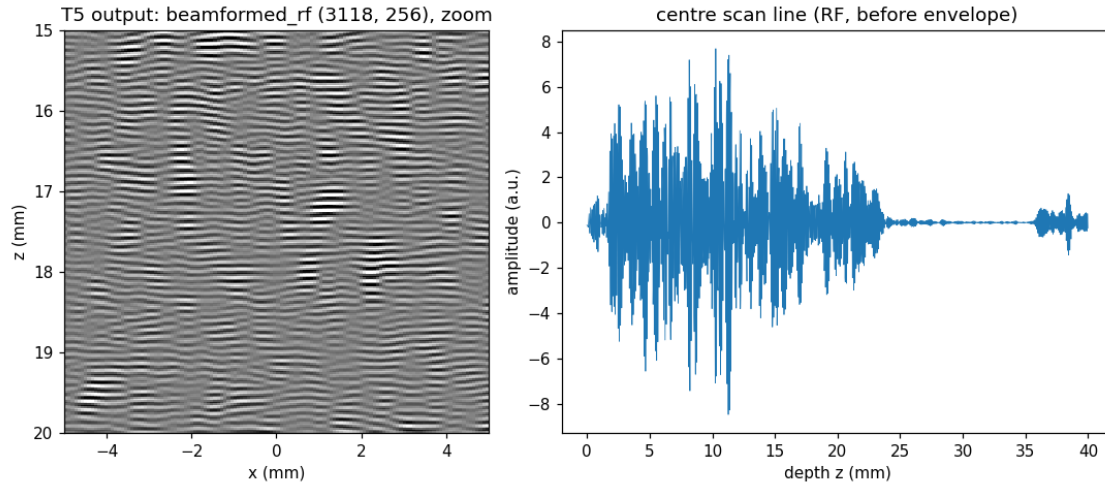
Image of the point at (0, 20) mm (point spread function) and its profiles. The red line is -6 dB.

Quantity	Measured	What controls it
Axial -6 dB width	0.327-0.346 mm	pulse length (theory 0.340 mm)
Lateral -6 dB width	0.471 mm	$\lambda \times F\#$, apodization window
Position error	≤ 0.04 mm	correct delays + time-axis rule
Highest side lobe (Hann)	-32.7 dB	window shape (rect -18.2 dB, Hamming -33.3 dB)

Worked example with our numbers

Frequency test (fs 128 MHz): axial width 0.674 / 0.327 / 0.212 mm at 4 / 8 / 12 MHz (theory 0.680 / 0.340 / 0.227) and lateral 0.820 / 0.460 / 0.340 mm: both shrink as frequency rises, as λ does.

T5's own verification (4 tests) passes: depth errors 0.008–0.040 mm at 15, 40 and 55 mm.



T5's output beamformed_rf (3118 × 256): zoom 15–20 mm, and the centre line before envelope detection.

Receives (input)	Does	Produces (output)	Code
raw_rf, t_axis, element positions, 256 scan-line positions	per line and depth: delay, Hann weight over the F# aperture, sum	beamformed_rf (3118, 256) + z_axis (0–60 mm)	receive_beamforming.das_beamform()

Check yourself

Q. What are the two steps of DAS?

A. Delay each channel by its travel time to the point, then add them (with weights).

Q. Why does the aperture grow with depth?

A. To keep the F-number constant, so focusing quality is similar at all depths, and to avoid steep angles near the surface.

Q. What trade-off does the Hann window make?

A. Much lower side lobes (−32.7 vs −18.2 dB) for a wider main lobe (0.46 vs 0.30 mm).

Q. Why is the output still RF, not an image?

A. It still oscillates at 8 MHz; T6 takes the envelope next.

8 T6 · Image reconstruction

Envelope detection and log compression

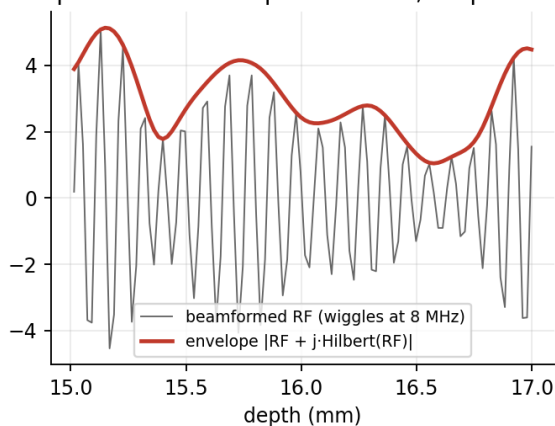
What this team does: turns each oscillating beamformed line into a smooth brightness profile, then compresses its huge range of values so that weak and strong echoes can be shown together.

8.1 Envelope detection with the Hilbert transform

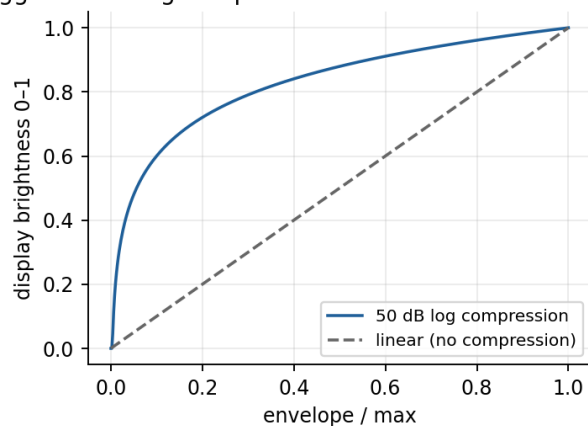
The beamformed RF wiggles up and down 8 million times a second. For a picture we only need its **outline** (envelope). The Hilbert transform $\mathcal{H}\{s\}$ makes a copy of the signal shifted by a quarter wave. Combining the two gives the **analytic signal**, whose magnitude is the envelope. It is smooth even where the RF crosses zero:

$$E(z) = |s(z) + j\mathcal{H}\{s\}(z)|$$

Envelope detection: keep the outline, drop the wiggles



Log compression lifts weak echoes



Left: real beamformed RF on the centre line and its envelope. Right: the 50 dB log-compression curve.

8.2 Log compression

Echoes range from very strong (a bright boundary) to very weak (deep speckle), a ratio of 1000 : 1 or more. The eye cannot see that range on a screen. Taking the logarithm (decibels) squeezes it:

$$L(z) = 20 \log_{10} \left(\frac{E(z)}{\max E} \right) \text{ [dB]}, \quad 0 \text{ dB} = \text{brightest point}$$

Worked example with our numbers

An echo 1/100 of the peak: $20 \log_{10}(0.01) = -40$ dB. Shown on a linear scale it would be 1 % gray (invisible); after 50 dB log compression it becomes $(-40 + 50)/50 = \mathbf{20\% \text{ gray}}$, clearly visible.

How our code does it

bmode_formation.py: `scipy.signal.hilbert` along depth $\rightarrow |\cdot| \rightarrow$ divide by the frame maximum $\rightarrow 20 \log_{10}$ (values below 10^{-12} are clipped to avoid log 0).

Same algorithm as the team's real-carotid notebook; on the real carotid RF it matches the team's saved output to 5×10^{-6} .

Receives (input)	Does	Produces (output)	Code
beamformed_rf (3118, 256)	Hilbert envelope, normalise to peak, $20 \log_{10}$	log-compressed envelope (3118, 256) in dB, 0 dB = peak	teams/T6_B_mode/bmode_formation.py

Check yourself

Q. Why can't we display the RF directly?

A. It oscillates positive and negative at 8 MHz; we need its magnitude outline (envelope).

Q. What does the Hilbert transform add?

A. A 90°-shifted copy; together they give the analytic signal whose magnitude is the envelope.

Q. Why logarithm?

A. It compresses a huge ratio of echo strengths into a range the eye can see.

9 T7 · B-mode image formation

Dynamic range, gray mapping and scan conversion

What this team does: turns the dB values into a gray-scale picture with real millimetre axes and square pixels.

9.1 Dynamic range and gray mapping

We choose to show a **50 dB dynamic range**: 0 dB (peak) is white, −50 dB and weaker is black, and everything in between is a shade of gray:

$$B = \frac{\text{clip}(L, -DR, 0) + DR}{DR} \in [0, 1], \quad DR = 50 \text{ dB}$$

Key idea: choosing the dynamic range

A larger DR shows weaker echoes but makes the background gray and noisy; a smaller DR gives more contrast but hides weak tissue. We kept 50 dB (T3's setting). The source of the carotid data displays it at 40 dB; the B-mode notebook used 60 dB without a source.

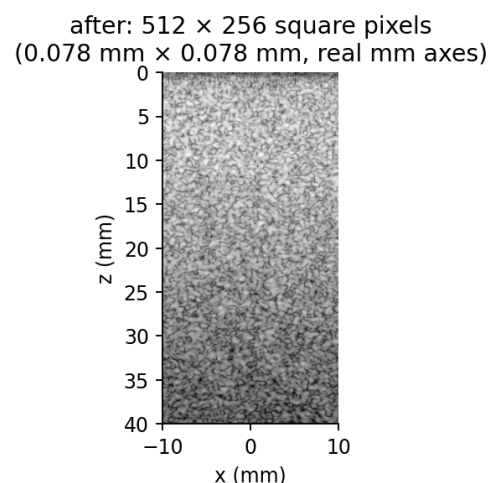
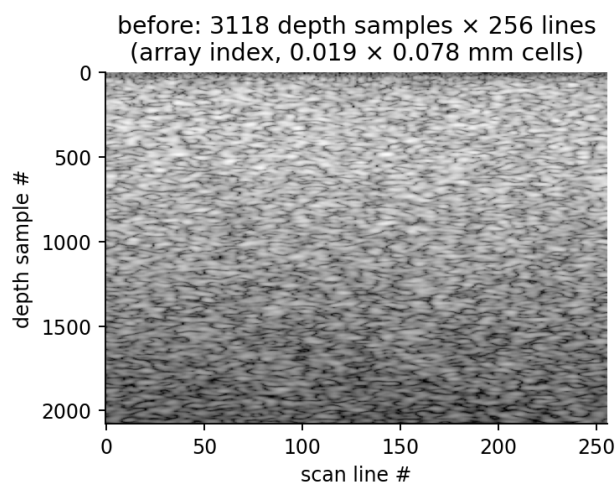
9.2 Scan conversion

After beamforming, the data is a table: 3118 depth samples × 256 lines, with cells 0.019 mm tall and 0.078 mm wide. A screen needs square pixels on a regular grid. **Scan conversion** resamples the table onto that grid.

- A **linear array** has parallel vertical lines, so this is a simple rectangular resampling. (Phased and curved arrays have fan-shaped lines and need polar-to-Cartesian conversion.)
- The depth is cropped to 0–40 mm, and every new pixel is computed by **bilinear interpolation**: a weighted average of the 4 nearest table values.
- Pixels are square (0.078 × 0.078 mm), so circles stay circles and the filter window in T6 is the same size in both directions.

$$f(x, z) \approx (1 - u)(1 - v)f_{00} + u(1 - v)f_{10} + (1 - u)v f_{01} + uv f_{11}$$

u, v = fractional position between the 4 neighbours



Before: a data table indexed by sample and line. After: 512 × 256 square pixels with mm axes (0.078 mm per pixel).

How our code does it

Gray map: the last step of `bmode_formation()` (clip to -50 dB, rescale to 0–1).

Scan conversion: `post_processing.scan_convert()` with `scipy`'s `RegularGridInterpolator` (linear) onto a square-pixel grid.

Receives (input)	Does	Produces (output)	Code
log-compressed envelope (T6), scan-line x-positions, z_axis	50 dB clip, 0–1 gray map, crop to 40 mm, bilinear resample	B-mode gray image (512, 256), 0.078 mm/px, values 0–1	<code>bmode_formation.py</code> , <code>post_processing.scan_convert()</code>

Check yourself

Q. What gray value does an echo at -25 dB get with $DR = 50$ dB?

A. $(-25 + 50)/50 = 0.5$, mid-gray.

Q. Why is scan conversion simple for our probe?

A. A linear array's lines are parallel, so no fan (polar) geometry is needed.

Q. Why square pixels?

A. So distances and shapes are the same in x and z, and the filter window is isotropic in mm.

10 T6 · Post-image processing

Despeckling with the guided filter

What this team does: reduces the grainy speckle so that tissue regions look smoother, **without** blurring the edges of structures such as the cyst.

10.1 Why not just blur?

An ordinary average (box or Gaussian blur) does smooth speckle, but it smooths everything, including the edges we want to see. An **edge-preserving** filter smooths only where the image is flat.

10.2 The guided filter (He, Sun and Tang, 2013)

Inside each small window (radius r , so $(2r+1) \times (2r+1)$ pixels), the filter assumes the output is a straight-line function of the guide image I : $q = a \cdot I + b$. It chooses a and b to stay close to the input while keeping a small:

$$a_k = \frac{\text{cov}_k(I, p)}{\text{var}_k(I) + \varepsilon}, \quad b_k = \bar{p}_k - a_k \bar{I}_k, \quad q = \bar{a} I + \bar{b}$$

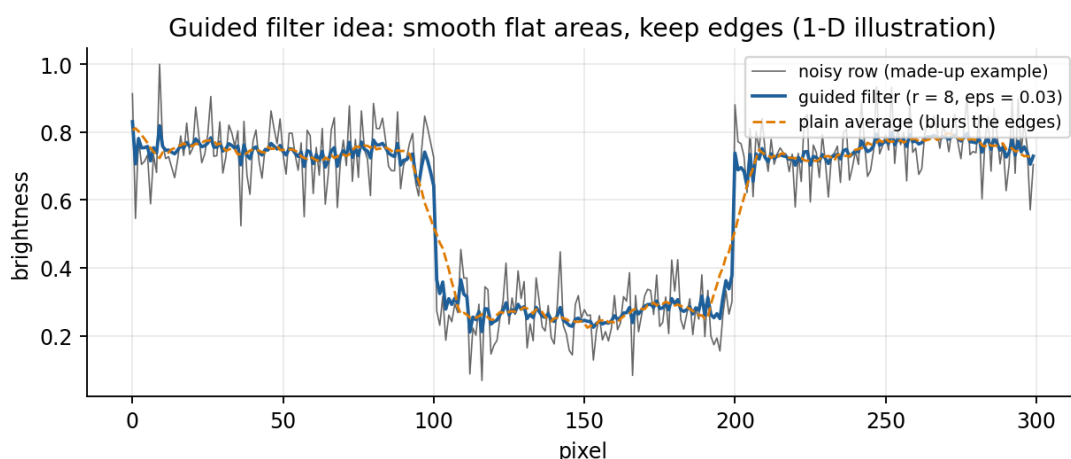
our code is self-guided ($I = p = \text{the image}$); bars are box-filter means over the window

Key idea: what ε does

In a **flat, speckled** region the local variance is small compared with ε , so $a \approx 0$ and $q \approx$ the local mean: **smoothing**.

At an **edge** the local variance is large compared with ε , so $a \approx 1$ and $q \approx I$: the **edge is kept**.

Bigger $\varepsilon \rightarrow$ more smoothing (and eventually softer edges). Bigger $r \rightarrow$ larger smoothing window.

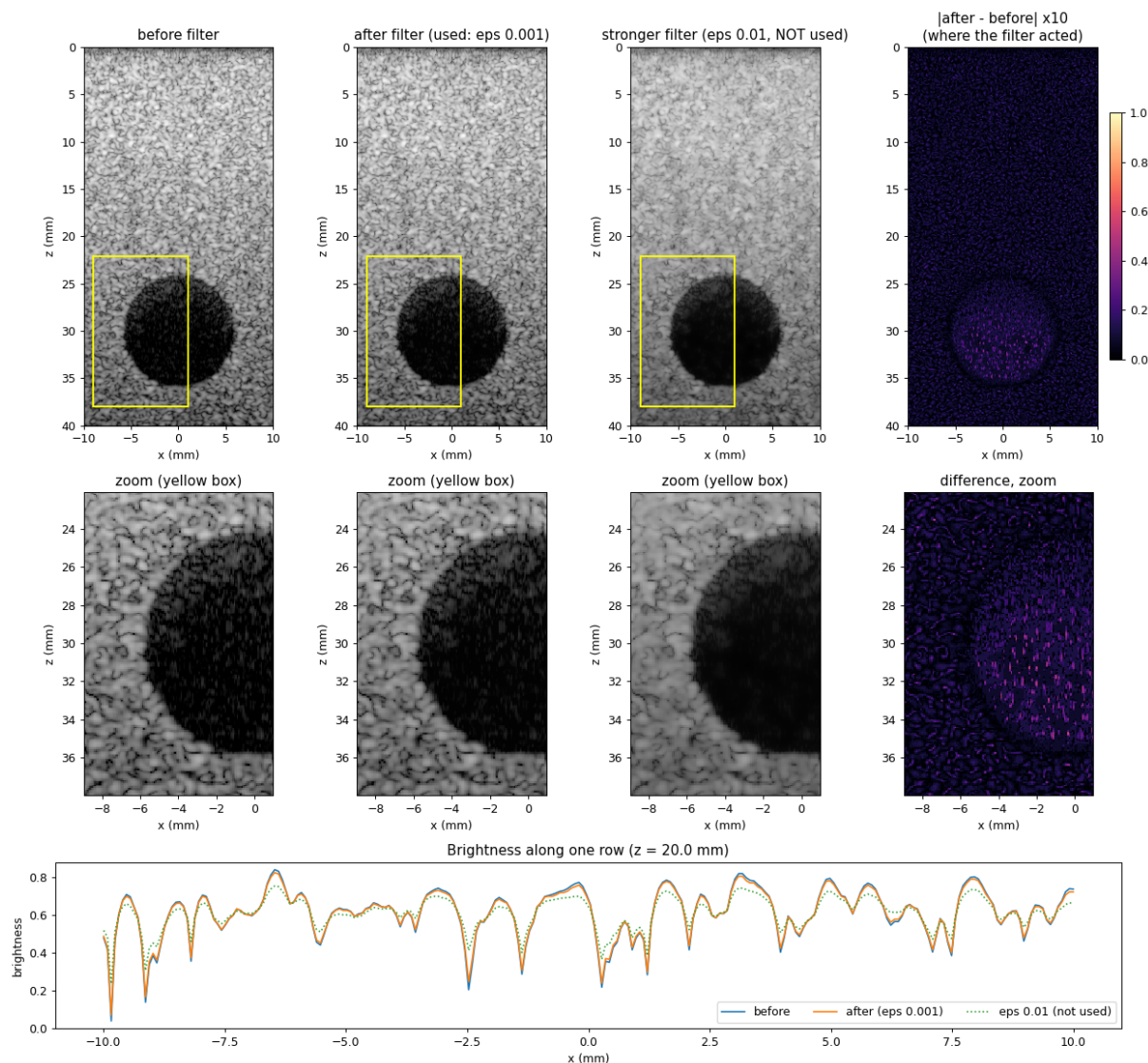


1-D illustration (made-up data): the guided filter smooths the flat parts but keeps the steps; a plain average blurs them.

10.3 Our settings and their effect

Our settings are $r = 4$ (9×9 window) and $\varepsilon = 0.001$ on intensities scaled to 0–1. They are from the team's post-processing notebook, where they were tuned on real breast images.

ϵ	Edge sharpness kept	gCNR (cyst)	Notebook rule: edge ≥ 0.90
0.001 (used)	0.96	0.871	pass
0.002	0.93	0.881	pass (an option for the team)
0.005	0.86	–	fail
0.01	0.78	–	fail



Before and after the filter on our cyst image: zoom, difference, one row, and histogram.

How our code does it

`post_processing.postprocess(image, method='guided', r=4, eps=0.001)`: float image $\rightarrow$ 8-bit gray (0-255) $\rightarrow$ the notebook's `guided()` (box means with `cv2.blur`) $\rightarrow$ back to 0-1 $\rightarrow$ optional gamma (1.0 = unchanged).

Receives (input)	Does	Produces (output)	Code
B-mode gray image (512, 256)	self-guided filter, $r = 4$, $\epsilon = 0.001$	final_bmode (512, 256), values 0-1	<code>post_processing.postprocess()</code>

Check yourself

Q. Why use a guided filter and not a Gaussian blur?

A. It smooths flat speckle regions but keeps edges, because $a \approx 1$ where the local variance is large.

Q. What happens if ϵ is made much larger?

A. More smoothing, but edges soften: at $\epsilon = 0.01$ only 78 % of the edge sharpness is kept.

Q. Does despeckling add information?

A. No. It makes regions easier to see; the underlying echo data is unchanged.

11 Measuring image quality

How we prove the images are right

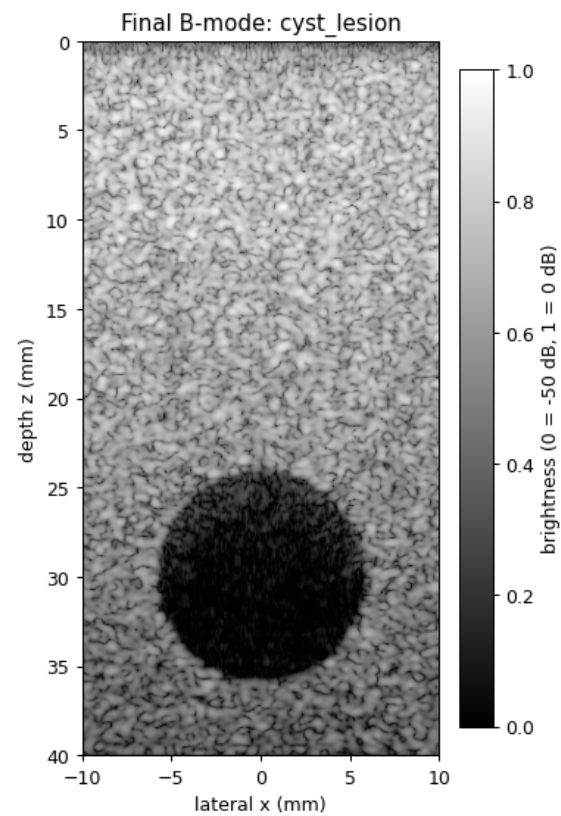
Every claim in the project is backed by a number measured on the images. These are the measurements, in plain words.

Metric	Simple meaning	Formula in our code	Better when
−6 dB width (axial / lateral)	size of the blur of a single point	width where the envelope is $\geq$ half of its peak	smaller
Speckle SNR	is the speckle realistic?	mean(E) / std(E) of the envelope	close to 1.91
Speckle index SI	how grainy the image is	mean over the image of local std / local mean (7×7)	smaller
Contrast	how dark the cyst is	mean(lesion) − mean(ring), in displayed dB	more negative
CNR	contrast compared with the graininess	$ \mu_L - \mu_B / \sqrt{(\sigma_L^2 + \sigma_B^2)}$	larger
gCNR	how separable lesion and background pixels are (0 = identical, 1 = fully separable)	$1 - \sum \min(h_L, h_B)$ (histogram overlap)	closer to 1
Edge sharpness	how crisp the cyst border is	mean Sobel gradient on a 3-px band around the border	larger

Background “B” = a ring 10 pixels (0.78 mm) wide around the lesion. The gCNR follows Rodriguez-Molares et al. (2020).

11.1 Our final results

Phantom	Result
Point targets	found at (−4.98, 10.01), (−0.04, 20.00), (+4.98, 29.99) mm; axial 0.33–0.35 mm; lateral 0.471 mm
Speckle	SNR 1.73–1.92 (theory 1.91); uniform across ± 10 mm within 0.2 dB; SI 0.178 → 0.163 after the filter
Cyst, before filter	contrast −19.40 dB, CNR 2.323, gCNR 0.860, SI 0.308
Cyst, after filter	contrast −19.32 dB, CNR 2.425, gCNR 0.871, SI 0.241, edge kept 96 %



Final B-mode image of the cyst phantom (50 dB, after the guided filter).

12 Parameter tests: theory checked by experiment

tests/parameter_tests.py · 9 of 9 checks pass

Each test changes one parameter, keeps everything else the same, and checks that the result moves the way theory predicts.

Parameter	Theory says	We measured	Verdict
Centre frequency 4 / 8 / 12 MHz	axial width = $(c/2) \cdot 2.355\sigma$: 0.680 / 0.340 / 0.227 mm; lateral width $\propto \lambda$	axial 0.674 / 0.327 / 0.212 mm; lateral 0.820 / 0.460 / 0.340 mm	✓ within 10 %
Attenuation $\alpha_0 = 0.5$ / 1.0	extra loss −16 / −32 dB (targets 10 vs 30 mm)	−16.11 / −32.21 dB	✓
Transmit focus 10 / 20 / 30 mm	strongest when focused at the target (20 mm)	+0.00 / +18.36 / +4.78 dB vs broadcast	✓
Receive window rect / Hamming / Hann	rect: narrowest beam, highest side lobes	0.300 / 0.420 / 0.460 mm; −18.2 / −33.3 / −32.7 dB	✓
Sampling 32 / 64 / 128 MHz	position unchanged (time-axis rule); level converges	24.987 / 25.006 / 25.006 mm; +0.00 / +2.79 / +3.39 dB	✓

Honest note

Two first versions of these tests failed, and were corrected only after the cause was measured: linear interpolation lowers the level at low sampling rates (so the test now checks convergence), and 40 MHz under-samples 12 MHz for a width measurement (so the frequency test runs at 128 MHz). Details: docs/MODULES.md, Section 11.

13 Assumptions and limitations

What our model does not include

Assumption	What it means	Effect
2-D model	no element height, no out-of-plane echoes	real images have some extra blur and clutter
Straight rays, one speed of sound	no refraction, fat/muscle speed differences	real images can have small position errors
Linear, point scatterers	no nonlinear (harmonic) propagation, no specular boundaries	no tissue-harmonic imaging
One broadcast transmit	focusing only on receive	transmit focus would add ~18 dB at its depth
No TGC	no depth gain correction	at 8 MHz the image is 17.2 dB darker at 36–40 mm than at 8–12 mm
Probe values	pitch 0.30 mm and f_0 8 MHz are T2's assumptions (Philips publishes 128 elements, 4–12 MHz)	resolution numbers depend on them
Phantoms, not organs	points, speckle and a cyst; no anatomy	a real body model would need CT/MRI tissue maps
Residual artefact	a faint –25 dB diagonal arc near the (–5, 10) mm target	not fully explained (documented)

TGC (time-gain compensation) is what real scanners do to undo attenuation: they amplify later echoes more. Our model does not add it, so deep regions look darker, which is physically correct for the raw signal.

14 Glossary

Words used in this project, in one line each

Term	Meaning
Anechoic	a region with no echoes; appears black (our cyst).
Apodization	weighting the elements (e.g. Hann) to reduce side lobes.
Aperture	the part of the array in use; full array = 38.4 mm.
Attenuation	loss of sound energy with distance; 0.5 dB/(MHz·cm).
Axial / lateral	along the beam (depth) / across it (sideways).
B-mode	brightness mode: echo strength shown as gray level.
Beamforming	combining element signals to transmit or listen in one direction/point.
Broadcast transmit	all elements fire together with no delays.
CNR / gCNR	contrast-to-noise ratio / generalized CNR (0–1, lesion detectability).
DAS	delay-and-sum beamforming.
dB	decibel, $20 \log_{10}$ of an amplitude ratio.
Directivity	how an element's sensitivity changes with angle (sinc pattern).
Dynamic range	span of dB values displayed (50 dB).
Dynamic receive focus	receive delays recomputed at every depth.
Envelope	smooth outline (magnitude) of an oscillating signal.
F-number (F#)	depth ÷ active aperture width; ours = 1.
Grating lobe	false copy of an object caused by pitch > $\lambda/2$.
Guided filter	edge-preserving smoothing filter used for despeckling.
Hilbert transform	90° phase-shifted copy of a signal, used to get the envelope.
Impedance (acoustic)	$Z = \text{density} \times \text{speed}$; changes in Z cause echoes.
Linear array	elements in a straight row with parallel scan lines.
Nyquist rule	sample at more than twice the highest frequency.
Phantom	a test object (points, speckle, cyst) that stands in for a patient.
Piezoelectric	material that converts voltage into vibration and back.
Pitch	centre-to-centre element spacing (0.30 mm).
RF signal	raw echo signal oscillating at MHz.
Scan conversion	resampling the line data onto a square-pixel display grid.
Scan line	one vertical image column formed by beamforming.
Side lobe	weaker beam beside the main beam; lowered by apodization.
Speckle	granular interference pattern from many sub-wavelength scatterers.
Speckle index (SI)	average local std / mean; lower = smoother.
TGC	time-gain compensation: extra gain for deeper echoes (not modelled).
Time of flight	travel time of sound = distance / c .

15 Formula sheet

Every equation of the pipeline on one page

Quantity	Equation
Wavelength	$\lambda = c/f_0$
Depth from time	$z = c t/2$
Element positions (T1)	$x_n = (n - \frac{N-1}{2})p$
Grating angle	$\sin \theta_g = \lambda/p$
Directivity	$D = \text{sinc}(w \sin \theta/\lambda)$
Pulse (T2/T3)	$p(t) = \sin(2\pi f_0 t) e^{-t^2/2\sigma^2}, \sigma = 3/(2f_0)$
Focus delays (T2)	$\tau_n = (\max_n d_n - d_n)/c$
Attenuation (T3)	$A = 10^{-\alpha_0 f_0 d/20}$
Reflection (T4)	$R = (Z_2 - Z_1)/(Z_2 + Z_1), G = \text{amp}(1 + R) \text{ scale}$
Speckle SNR	$\sqrt{\pi/(4 - \pi)} = 1.91$
DAS (T5)	$s(z) = \sum_m w_m \text{rf}_m(\tau_{TX} + \tau_{RX,m}) / \sum_m w_m$
Aperture (T5)	$a(z) = \max(z/2F_\#, 2p)$
Envelope (T6)	$E = s + j\mathcal{H}\{s\} $
Log compression (T6)	$L = 20\log_{10}(E/\max E)$
Gray map (T7)	$B = (\text{clip}(L, -50, 0) + 50)/50$
Guided filter (T6)	$a = \text{cov}/(\text{var} + \varepsilon), q = \bar{a}I + \bar{b}$
CNR	$ \mu_L - \mu_B /\sqrt{\sigma_L^2 + \sigma_B^2}$
gCNR	$1 - \sum \min(h_L, h_B)$

16 References and further reading

- [1] T. L. Szabo, *Diagnostic Ultrasound Imaging: Inside Out*, 2nd ed., Academic Press, 2014. (General textbook: propagation, attenuation, arrays, beamforming, B-mode.)
- [2] J. A. Jensen, "Field: A program for simulating ultrasound systems," *Medical & Biological Engineering & Computing*, 34 (Suppl. 1), 351–353, 1996. (Point-scatterer phantoms, element directivity.)
- [3] C. B. Burckhardt, "Speckle in ultrasound B-mode scans," *IEEE Trans. Sonics and Ultrasonics*, 25(1), 1–6, 1978.
- [4] R. F. Wagner, S. W. Smith, J. M. Sandrik and H. Lopez, "Statistics of speckle in ultrasound B-scans," *IEEE Trans. Sonics and Ultrasonics*, 30(3), 156–163, 1983. (Rayleigh statistics, SNR 1.91.)
- [5] K. He, J. Sun and X. Tang, "Guided image filtering," *IEEE Trans. Pattern Analysis and Machine Intelligence*, 35(6), 1397–1409, 2013.
- [6] A. Rodriguez-Molares et al., "The generalized contrast-to-noise ratio: a formal definition for lesion detectability," *IEEE Trans. Ultrasonics, Ferroelectrics and Frequency Control*, 67(4), 745–759, 2020.
- [7] Philips L12-4 (FUS4103) linear transducer: product information (128 elements, 4–12 MHz).
- [8] Project documents: docs/MODULES.md (per-module reference), docs/README.md (development log), docs/NITK-UsoundSim_Project_Flow.pdf, demo.ipynb.

How this guide was made

Figures: docs/source/make_theory_figures.py (from config.py, the teams' code and results/*.npz).
PDF: docs/source/make_theory_pdf.py. Measured numbers come from simulator/main.py, tests/parameter_tests.py and T5's verify_beamformer.py, as recorded in docs/MODULES.md.